



Parallel & Distributed Computing Virtualization-II Lecture 13/1

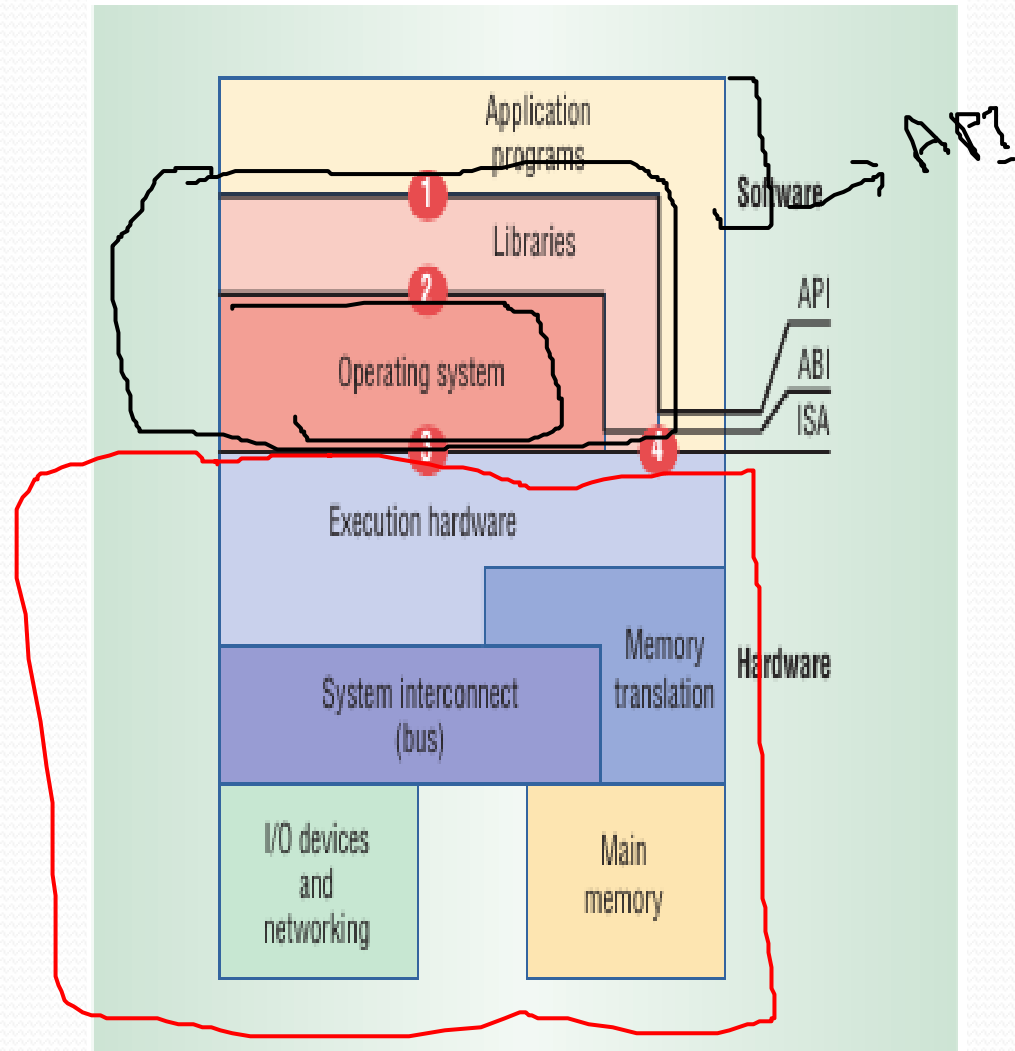
Farhad M. Riaz

Farhad.Muhammad@numl.edu.pk

**Department of Computer Science
NUML, Islamabad**

Computer System Architecture

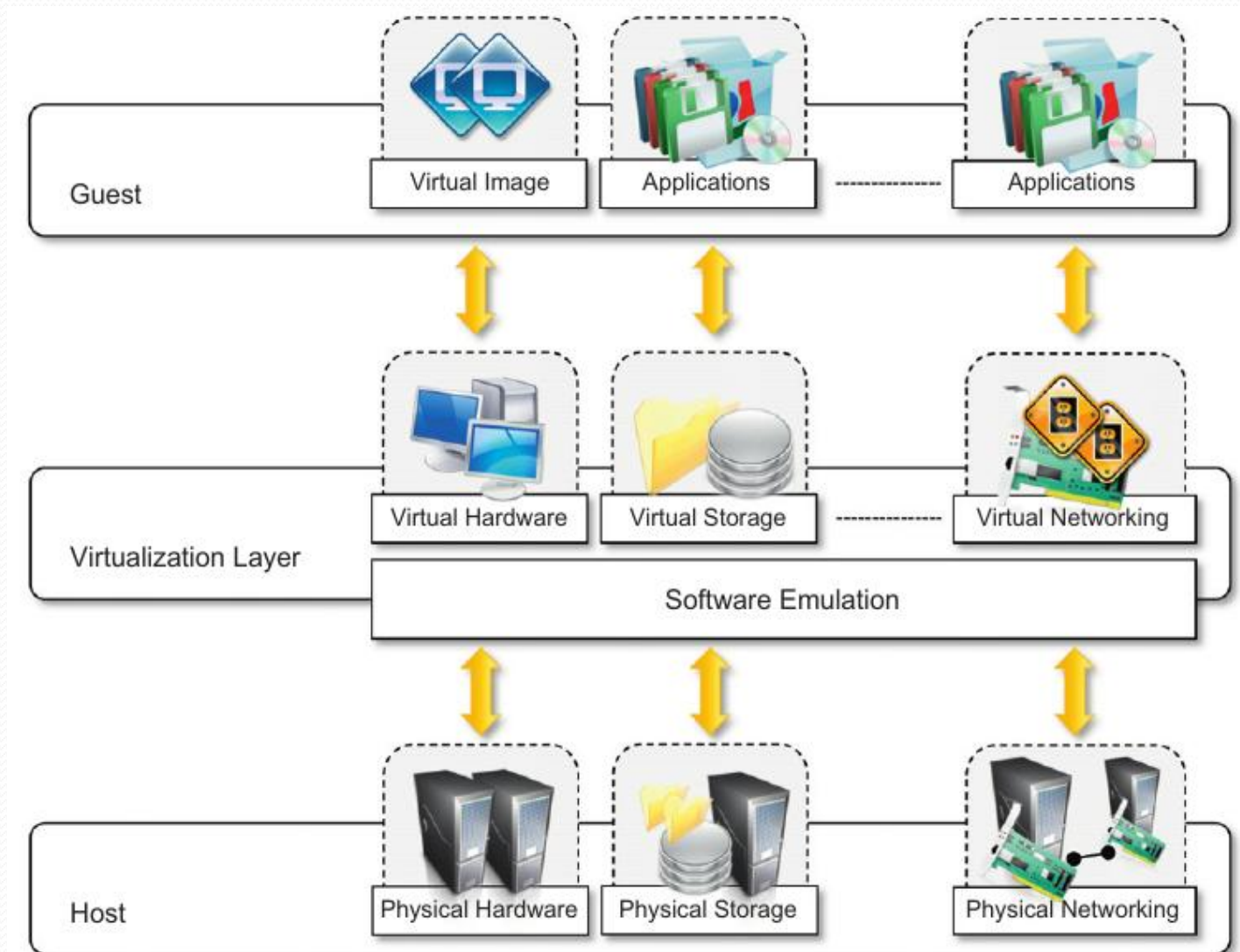
- Interfaces at or near the H/w S/w boundary :-
 - **ISA** – Instruction Set Architecture.
 - **API** – Application Program Interface
 - **ABI** – Application Binary Interface



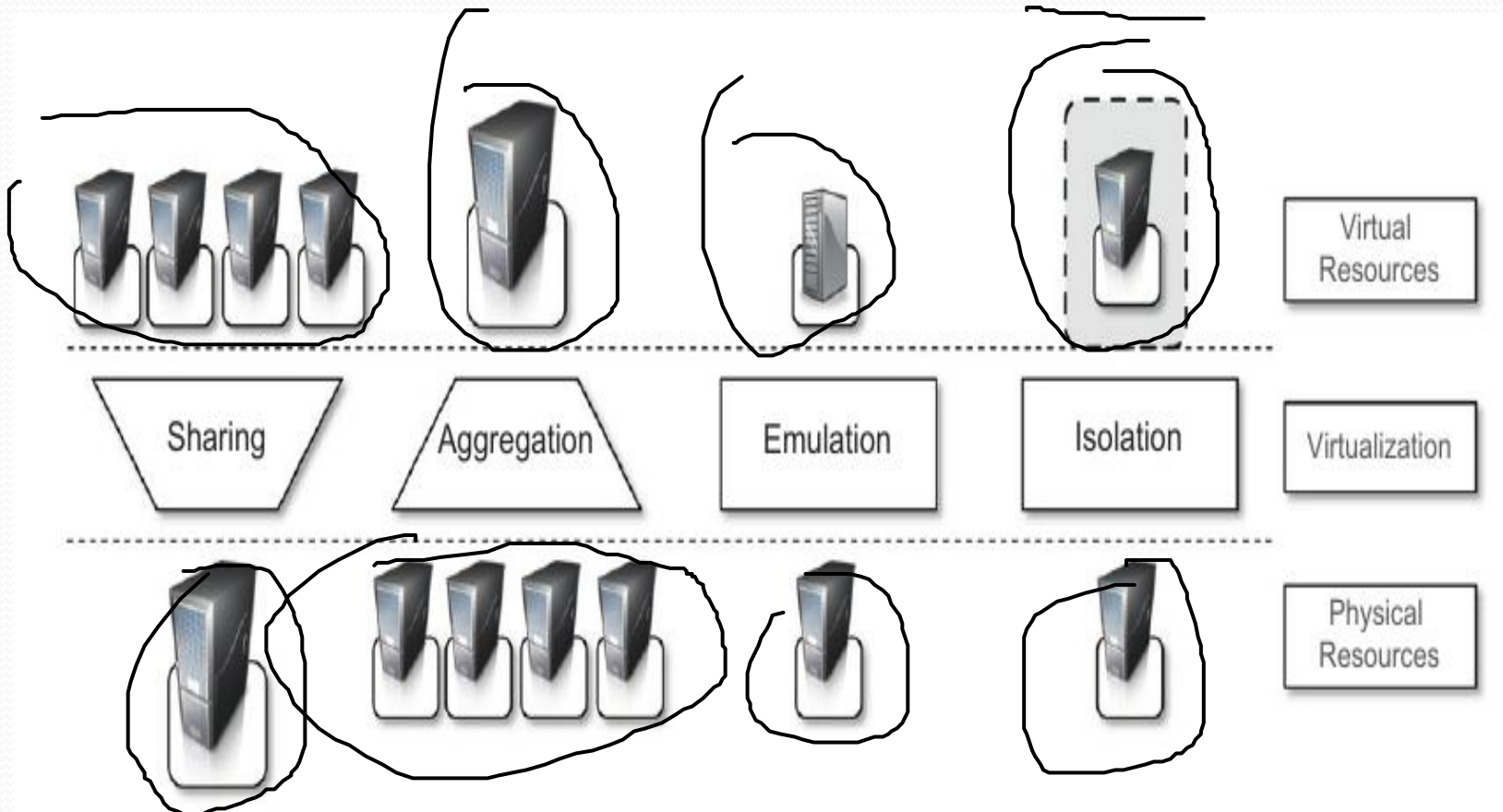
Virtualized Environments

- Three major components of Virtualized Environments
 - **Guest**
 - System component that interacts with Virtualization Layer.
 - **Host**
 - Original environment where guest runs.
 - **Virtualization Layer**
 - Recreate the same or different environment where guest will run.

Virtualization Reference Model



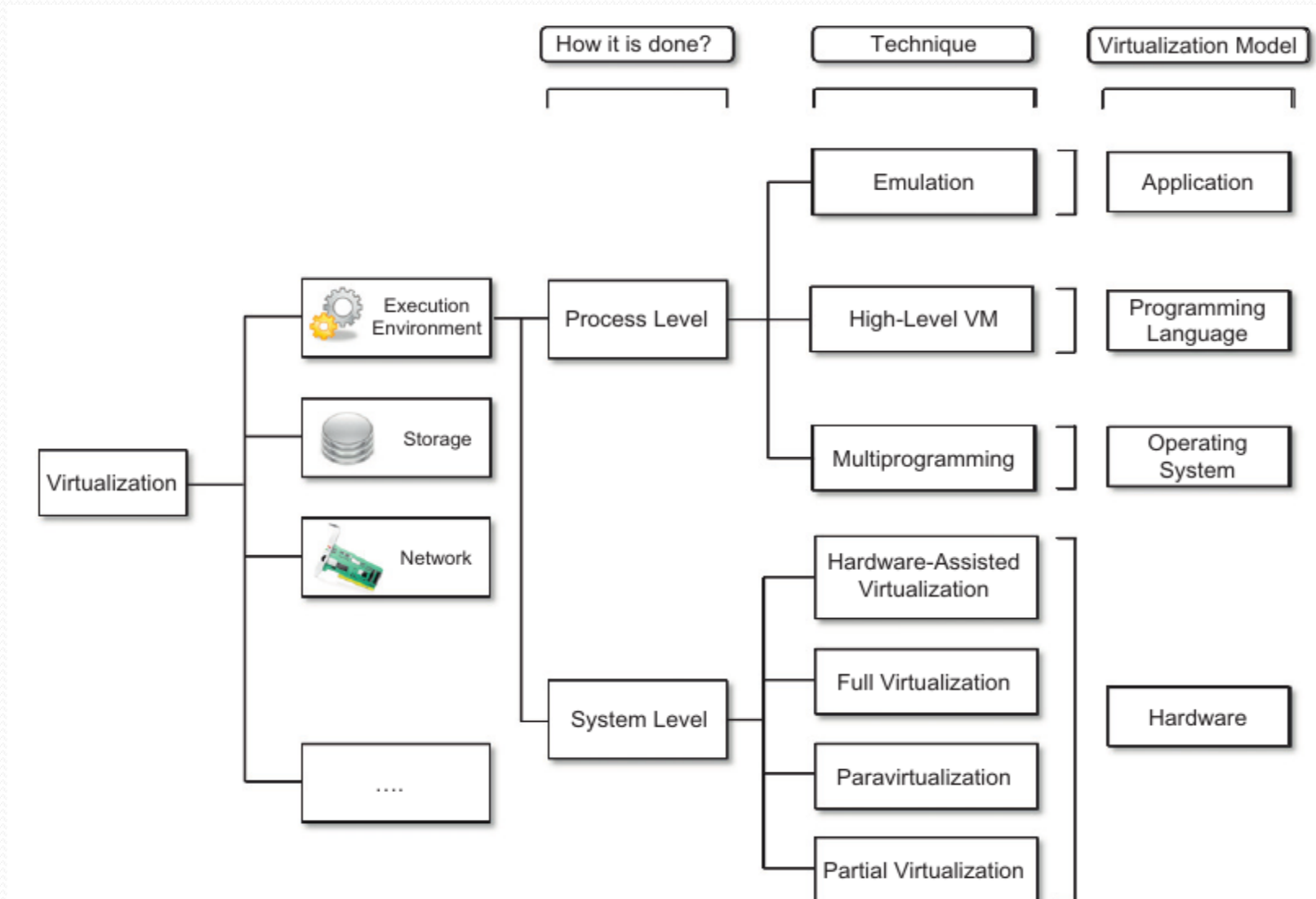
Managed Execution



Taxonomy of Virtualization Techniques

- Virtualization is mainly used to emulate **execution environment** , **storage** and **networks**.
- Execution Environment classified into two :-
 - **Process-level**
 - implemented on top of an existing operating system.
 - **System-level**
 - implemented directly on hardware and do not or minimum requirement of existing operating system

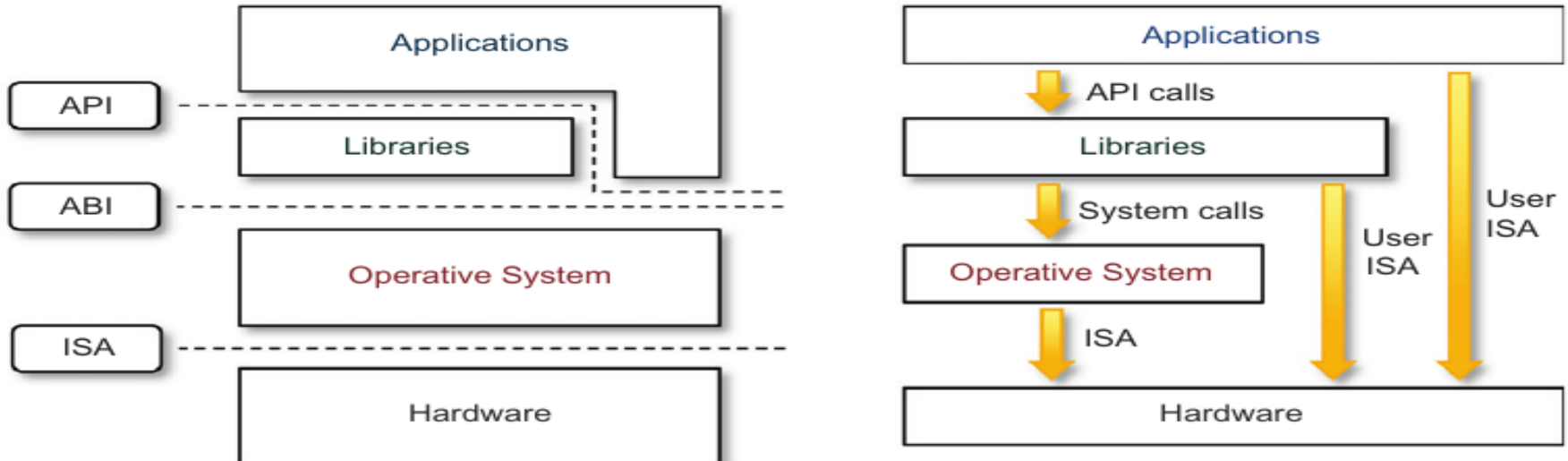
Taxonomy of virtualization



Machine Reference Model

- It defines the **interfaces between the levels** of abstractions, which **hide implementation details**.
- Virtualization techniques actually **replace one of the layers** and intercept the calls that are directed towards it.

Machine Reference Model [Cont.]



- Hardware is expressed in terms of the **Instruction Set Architecture (ISA)**.
 - ISA for processor, registers, memory and the interrupt management.
- **Application Binary Interface (ABI)** separates the OS layer from the application and libraries which are managed by the OS.
 - System Calls defined
 - Allows portabilities of applications and libraries across OS.

Machine Reference Model [Cont.]

- API – it interfaces applications to libraries and/or the underlying OS.
- Layered approach simplifies the development and implementation of computing system.
- ISA has been divided into two security classes:-
 - **Privileged Instructions**
 - **Nonprivileged Instructions**

ISA: Security Classes

- **Nonprivileged instructions**

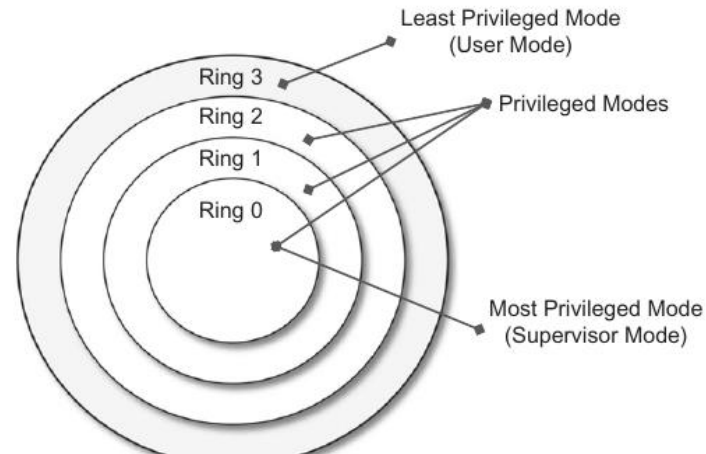
- That can be used without interfering with other tasks because they **do not access shared resources**.
- Ex. Arithmetic , floating & fixed point.

- **Privileged instructions**

- That are executed under **specific restrictions** and are mostly used for **sensitive operations**, which expose (behavior-sensitive) or modify (control-sensitive) the privileged state.
 - **Behavior-sensitive** – operate on the I/O
 - **Control-sensitive** – alter the state of the CPU register.

Privileged Hierarchy: Security Ring

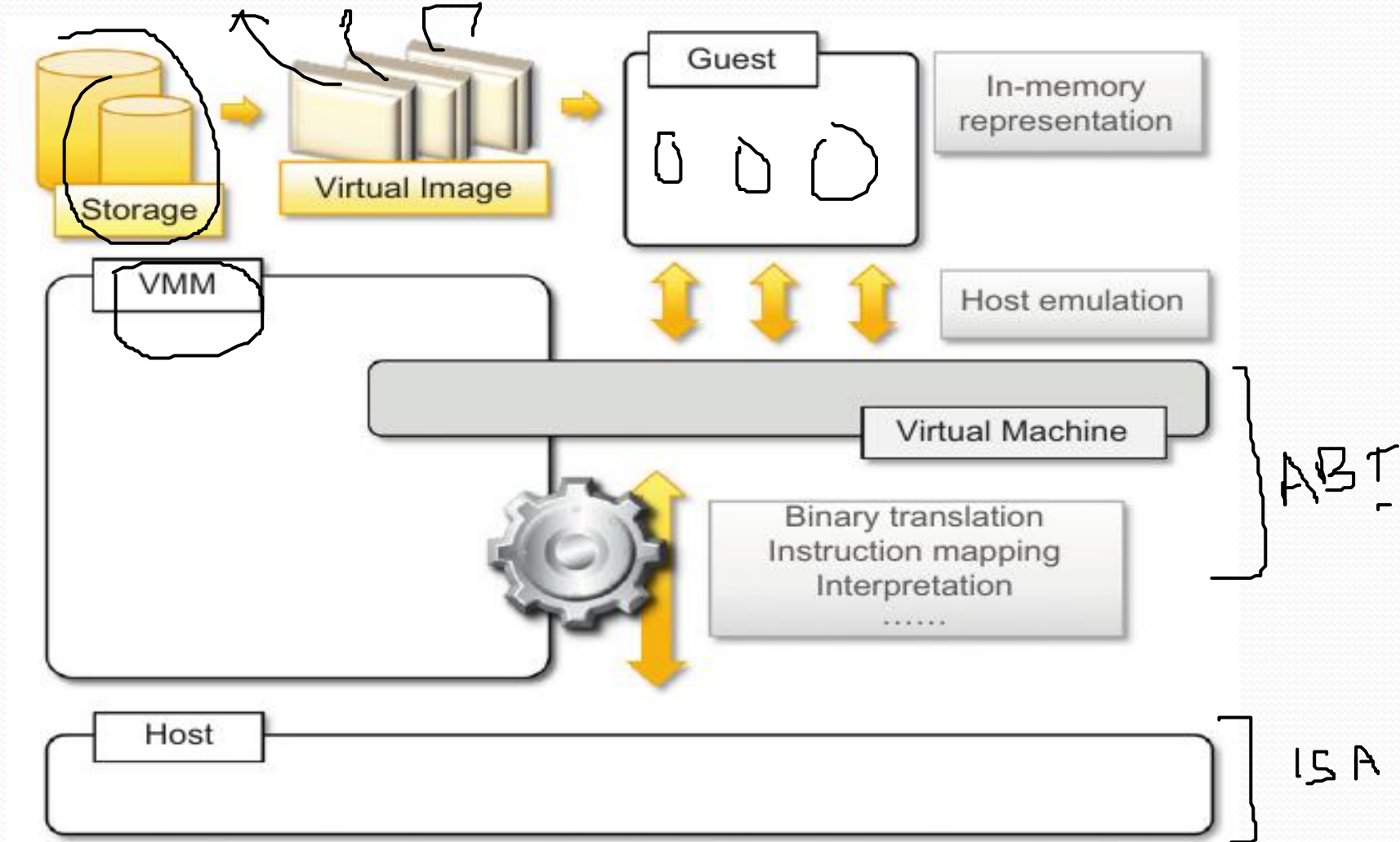
- Ring-0 is in most privileged level , used by the kernel.
- Ring-1 & 2 used by the OS-level services
- and , R3 in the least privileged level , used by the user.
- Recent system support two levels :-
 - **Ring 0 – supervisor mode**
 - **Ring 3 – user mode**



Hardware-level virtualization

- It is a virtualization technique that provides an abstract execution environment in terms of computer hardware on top of which a guest OS can be run.
- It is also called as system virtualization.

Hardware-level virtualization

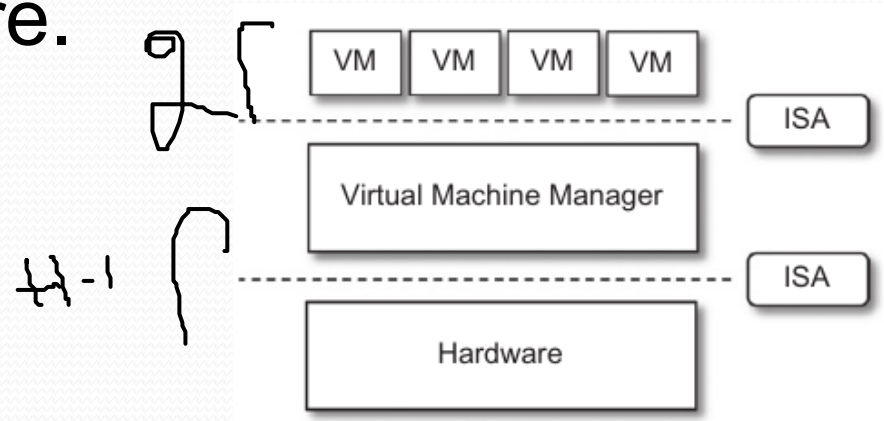


Hypervisor

- Hypervisor runs above the supervisor mode.
- It runs in supervisor mode.
- It recreates a h/w environment.
- It is a piece of s/w that enables us to run one or more VMs on a physical server(host).
- Two major types of hypervisor
 - **Type -I**
 - **Type-II**

Type-I Hypervisor

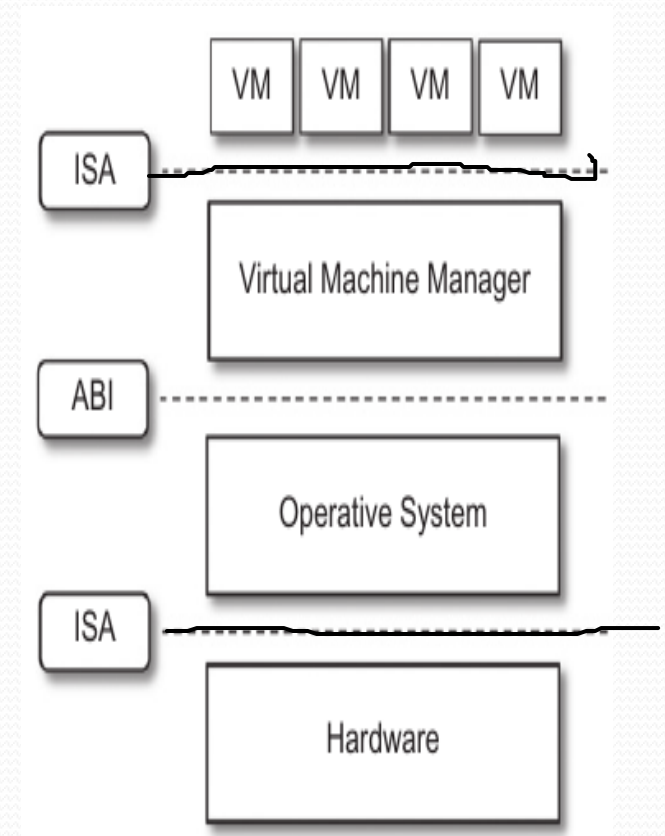
- It runs directly on top of the hardware.
- Takes place of OS.
- Directly interact with the ISA exposed by the underlying hardware.



- Also known as native virtual machine.

Type-II Hypervisor

- It requires the support of an operating system to provide virtualization services.
- Programs managed by the OS.
- Emulate the ISA of virtual h/w.
- Also called hosted virtual machine.



Virtual Machine Manager (VMM)

- Main Modules :-

- **Dispatcher**

- Entry Point of VMM
 - Reroutes the instructions issued by VM instance.

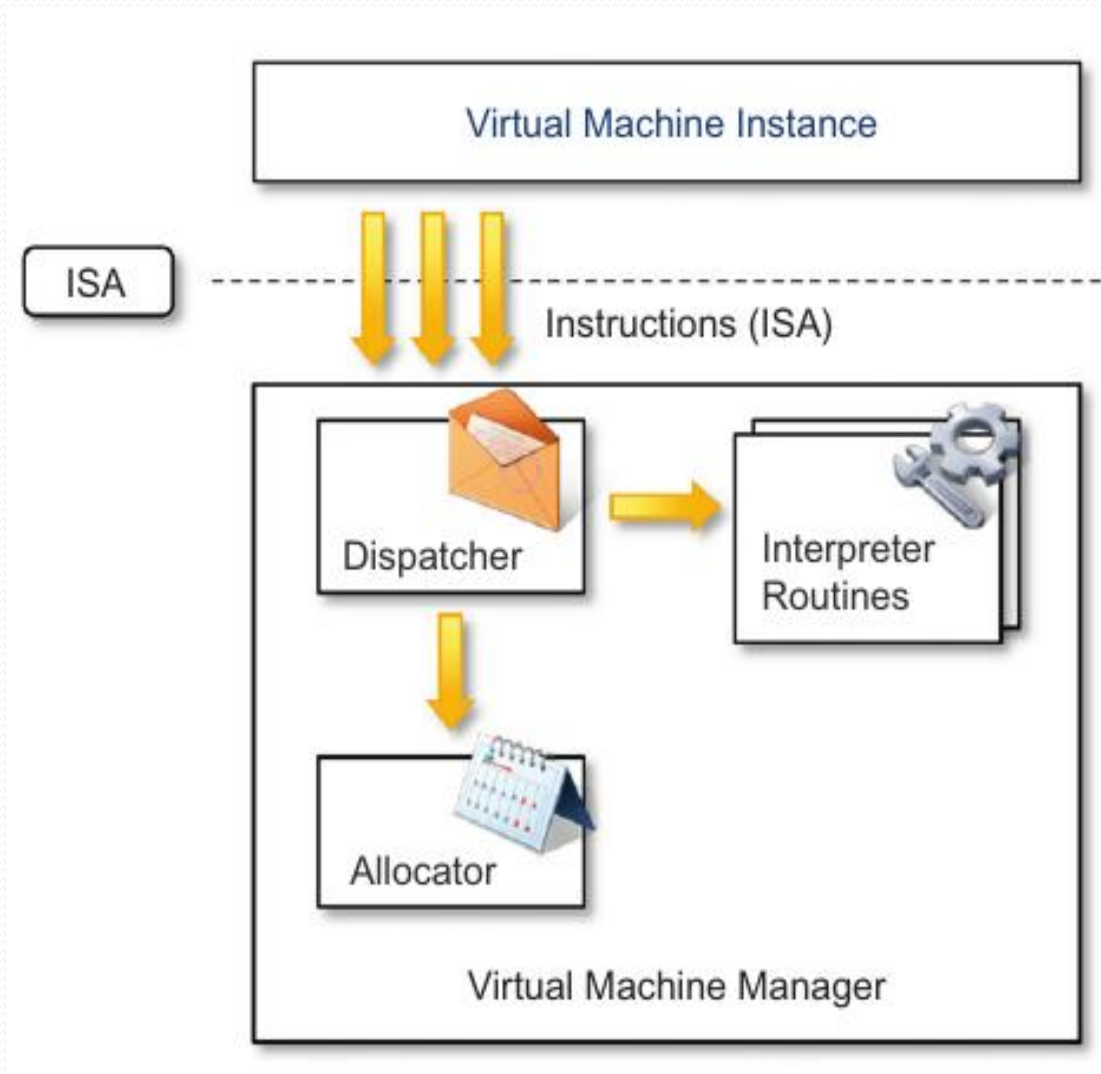
- **Allocator**

- Deciding the system resources to be provided to the VM.
 - Invoked by dispatcher

- **Interpreter**

- Consists of interpreter routines
 - Executed whenever a VM executes a privileged instruction.
 - Trap is triggered and the corresponding routine is executed.

Virtual Machine Manager (VMM)



Criteria of VMM

- Equivalence
 - same behaviour as when it is **executed directly** on the physical host.
- Resource control
 - it should be in **complete control** of virtualized **resources**.
- Efficiency
 - a statistically dominant fraction of the machine instructions should be **executed without intervention** from the VMM

What is containerization?

- Containerization is a software deployment process that bundles an application's code with all the files and libraries it needs to run on any infrastructure.
- Traditionally, to run any application on your computer, you had to install the version that matched your machine's operating system.
- For example, you needed to install the Windows version of a software package on a Windows machine.
- However, with containerization, you can create a single software package, or **container**, that runs on all types of devices and operating systems.

Containerization

Running Any App Anywhere



XENONSTACK

Anywhere

From Dev



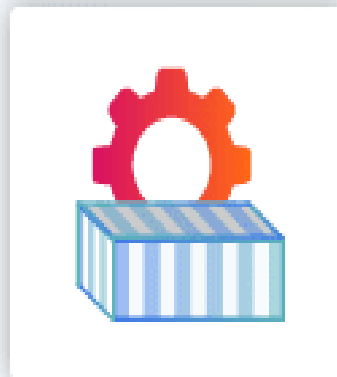
To Ops

Any App

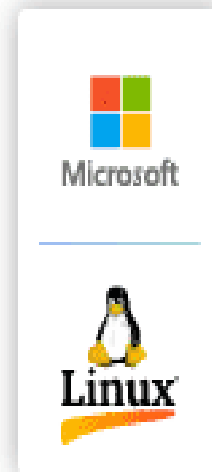


More

Containerization
Engine



Any OS



What are the benefits of containerization?

- Developers use containerization to build and deploy modern applications because of the following advantages.
- **Portability**
 - Software developers use containerization to deploy applications in multiple environments without rewriting the program code.
 - They build an application once and deploy it on multiple operating systems.
 - For example, they run the same containers on Linux and Windows operating systems.
 - Developers also upgrade legacy application code to modern versions using containers for deployment.

What are the benefits of containerization?

- **Scalability**

- Containers are lightweight software components that run efficiently.
- For example, a virtual machine can launch a containerized application faster because it doesn't need to boot an operating system.
- Therefore, software developers can easily add multiple containers for different applications on a single machine.
- The container cluster uses computing resources from the same shared operating system, but one container doesn't interfere with the operation of other containers.

What are the benefits of containerization?

- **Fault tolerance**

- Software development teams use containers to build fault-tolerant applications.
- They use multiple containers to run microservices on the cloud. Because containerized microservices operate in isolated user spaces, a single faulty container doesn't affect the other containers.
- This increases the resilience and availability of the application.

What are containerization use cases?

- **Cloud migration**

- Cloud migration, or the *lift-and-shift* approach, is a software strategy that involves encapsulating legacy applications in containers and deploying them in a cloud computing environment.
- Organizations can modernize their applications without rewriting the entire software code.

What are containerization use cases?

- **Adoption of microservice architecture**

- Organizations seeking to build cloud applications with microservices require containerization technology.
- The microservice architecture is a software development approach that uses multiple, interdependent software components to deliver a functional application.
- Each microservice has a unique and specific function.
- A modern cloud application consists of multiple microservices.
- For example, a video streaming application might have microservices for data processing, user tracking, billing, and personalization.
- Containerization provides the software tool to pack microservices as deployable programs on different platforms.

What are containerization use cases?

- **IoT devices**

- Internet of Things (IoT) devices contain limited computing resources, making manual software updating a complex process.
- Containerization allows developers to deploy and update applications across IoT devices easily.

How does containerization work?

- Containerization involves building self-sufficient software packages that perform consistently, regardless of the machines they run on.
- Software developers create and deploy container images—that is, files that contain the necessary information to run a containerized application.
- Developers use containerization tools to build container images based on the Open Container Initiative (OCI) image specification.
- OCI is an open-source group that provides a standardized format for creating container images.
- Container images are read-only and cannot be altered by the computer system.

How does containerization work?

- Container images are the top layer in a containerized system that consists of the following layers.
- **Infrastructure**
 - Infrastructure is the hardware layer of the container model.
 - It refers to the physical computer or bare-metal server that runs the containerized application.
- **Operating system**
 - The second layer of the containerization architecture is the operating system.
 - Linux is a popular operating system for containerization with on-premise computers.
 - In cloud computing, developers use cloud services such as **AWS EC2** to run containerized applications.

How does containerization work?

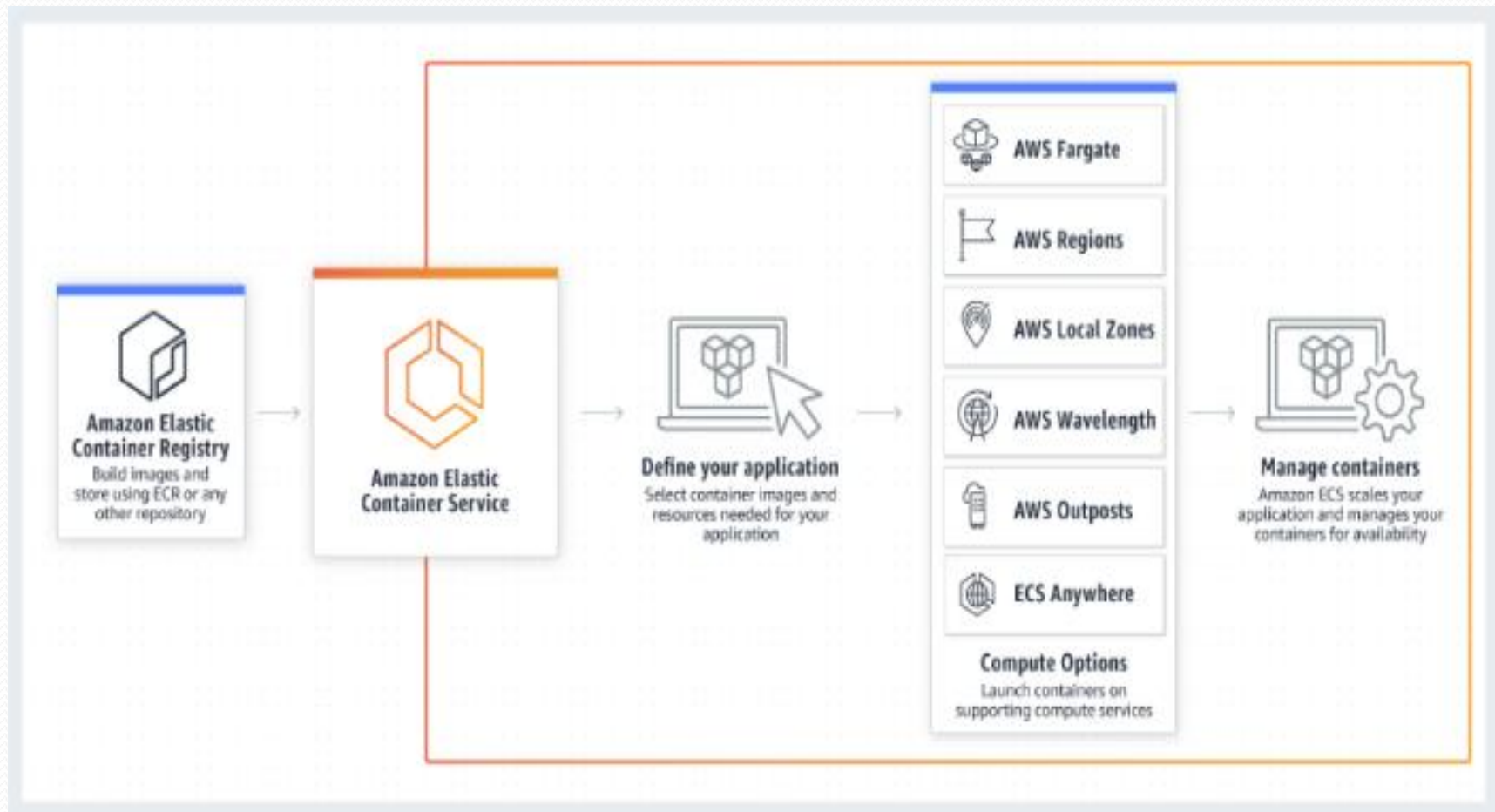
- **Container engine**

- The container engine, or container runtime, is a software program that creates containers based on the container images.
- It acts as an intermediary agent between the containers and the operating system, providing and managing resources that the application needs.
- For example, container engines can manage multiple containers on the same operating system by keeping them independent of the underlying infrastructure and each other.

- **Application and dependencies**

- The topmost layer of the containerization architecture is the application code and the other files it needs to run, such as library dependencies and related configuration files.
- This layer might also contain a light guest operating system that gets installed over the host operating system.

How does containerization work?



What are the types of container technology?

- The following are some examples of popular technologies that developers use for containerization.
- **Docker**
 - Docker, or Docker Engine, is a popular open-source container runtime that allows software developers to build, deploy, and test containerized applications on various platforms.
 - Docker containers are self-contained packages of applications and related files that are created with the Docker framework.

What are the types of container technology?

- **Linux**

- Linux is an open-source operating system with built-in container technology.
- Linux containers are self-contained environments that allow multiple Linux-based applications to run on a single host machine.
- Software developers use Linux containers to deploy applications that write or read large amounts of data.
- Linux containers do not copy the entire operating system to their virtualized environment.
- Instead, the containers consist of necessary functionalities allocated in the Linux namespace.

What are the types of container technology?

- **Kubernetes**

- Kubernetes is a popular open-source container orchestrator that software developers use to deploy, scale, and manage a vast number of microservices.
- It has a declarative model that makes automating containers easier.
- The declarative model ensures that Kubernetes takes the appropriate action to fulfil the requirements based on the configuration files.
-

Containerization compared to virtual machines

- Containerization is a similar but improved concept of a VM.
- Instead of copying the hardware layer, containerization removes the operating system layer from the self-contained environment.
- This allows the application to run independently from the host operating system.
- Containerization prevents resource waste because applications are provided with the exact resources they need.

What is serverless computing?

- Serverless computing refers to a cloud computing technology where the cloud vendor fully manages the server infrastructure powering an application.
- This means that developers and organizations do not need to configure, maintain, or provision resources on the cloud server.
- Serverless computing allows organizations to automatically scale computing resources according to the workload.

Containerization compared to serverless computing

- Serverless computing allows instant deployment of applications because there are no dependencies such as libraries or configuration files involved.
- The cloud vendor doesn't charge for computing resources when the serverless application is idle.
- Containers, on the other hand, are more portable, giving developers complete control of the application's environment.

What is cloud native?

- Cloud native is a software development method that builds, tests, and deploys an application in the cloud.
- The term *cloud native* means that the application is born and resides in a cloud computing environment.
- Organizations build cloud-native applications because they are highly scalable, resilient, and flexible.

Containerization compared to cloud native

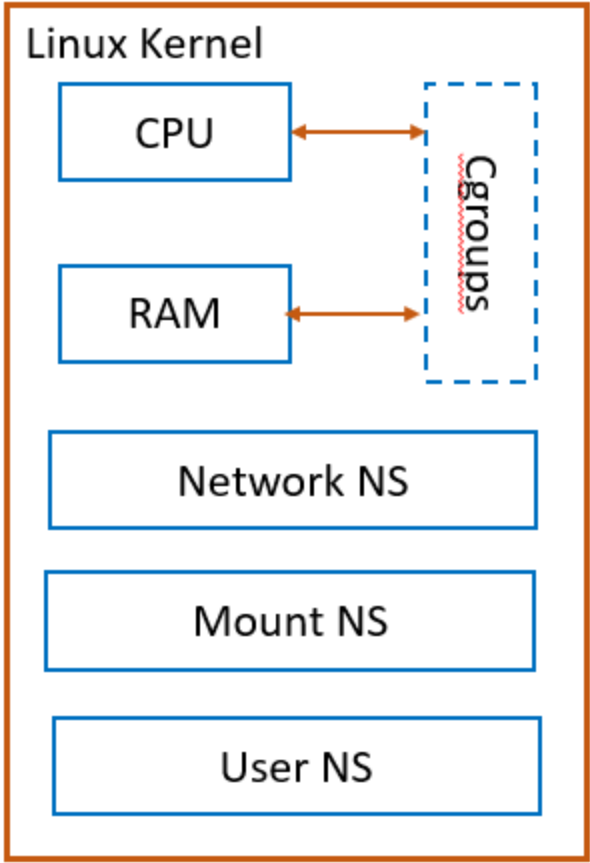
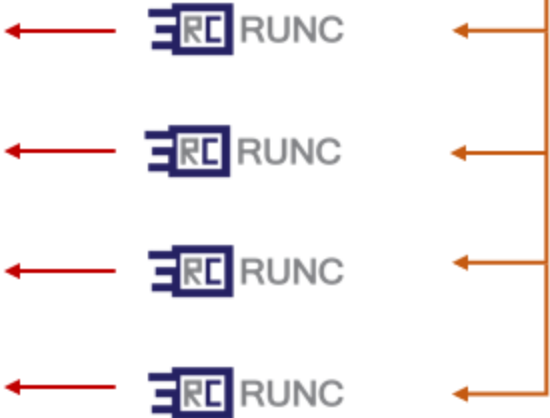
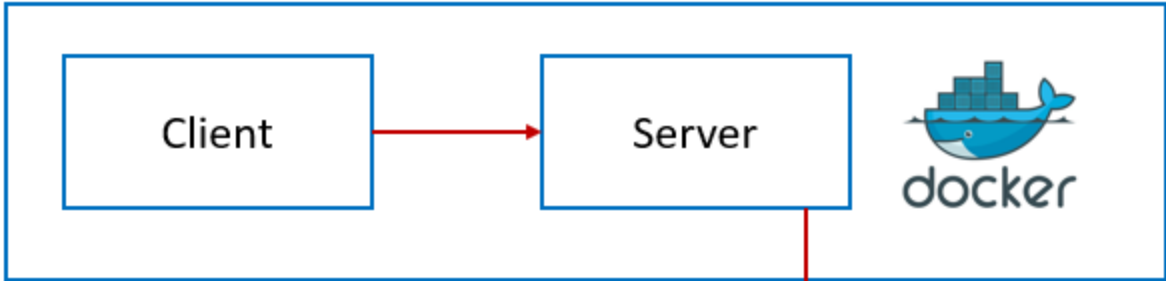
- Cloud-native application development requires different technologies and approaches than conventional monolithic applications.
- Containerization is one of the technologies that allows developers to build cloud-native applications.
- It works with other cloud-native technologies, such as service mesh and APIs, to allow microservices to work cohesively in a cloud-native application.

What is AWS App2Container?

- AWS App2Container is a containerization tool that allows software developers to modernize legacy applications.
- Developers use App2Container to turn Java and .NET applications into containerized applications:
- App2Container provides standard tools for migrating existing platform-based applications to the AWS environment.
- With App2Container, developers are assured of security when scaling cloud applications.
- Developers use App2Container to simplify identifying dependencies and configurations when creating container images.

What are Docker

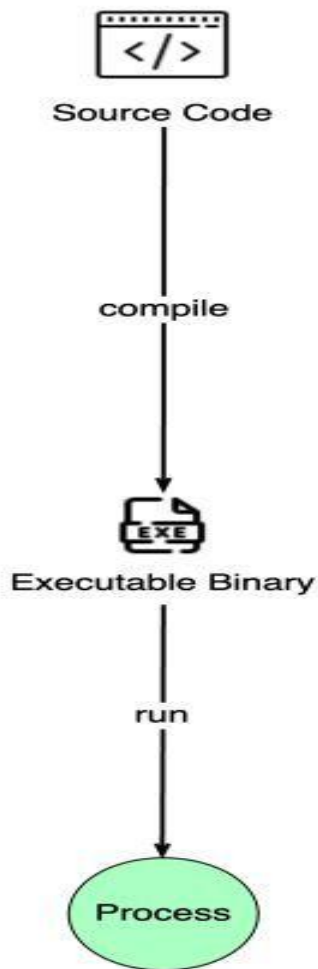
- Docker is a software platform that allows you to build, test, and deploy applications quickly.
- Docker packages software into standardized units called containers that have everything the software needs to run including libraries, system tools, code, and runtime.
- Using Docker, you can quickly deploy and scale applications into any environment and know your code will run.
- Running Docker on AWS provides developers and admins a highly reliable, low-cost way to build, ship, and run distributed applications at any scale.



Operating
System

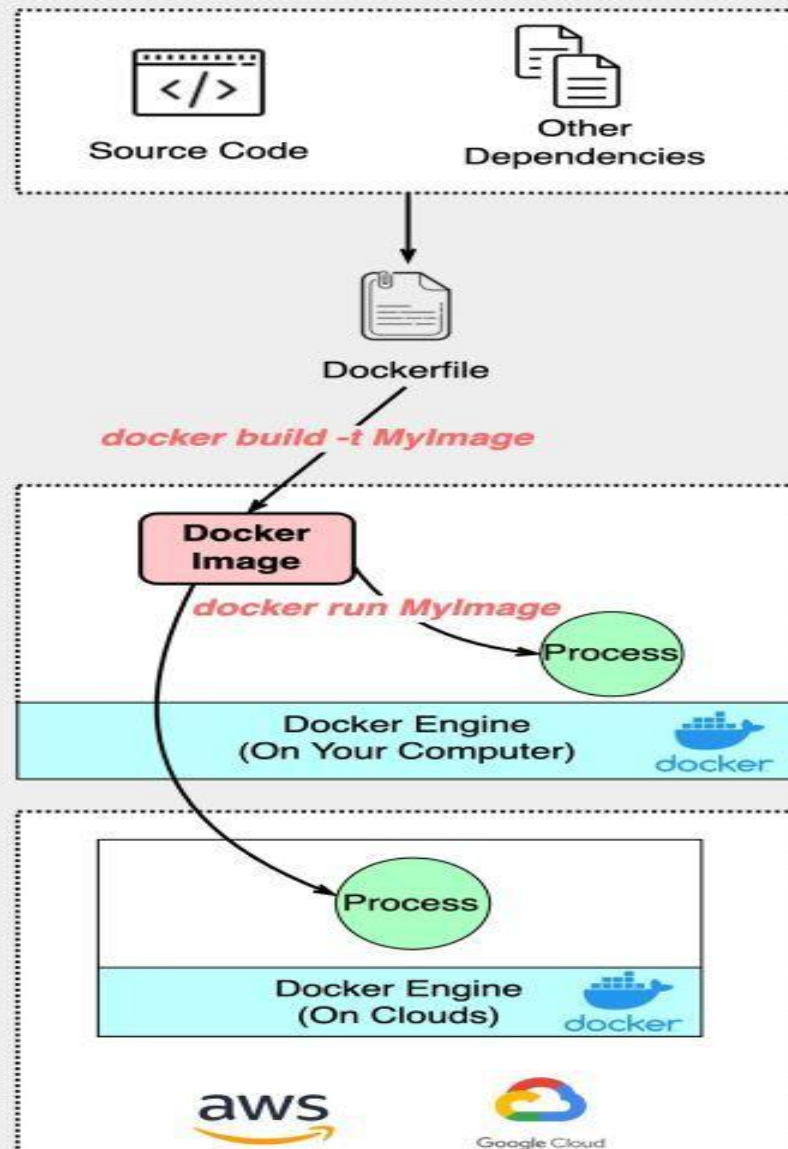
How does Docker Work?

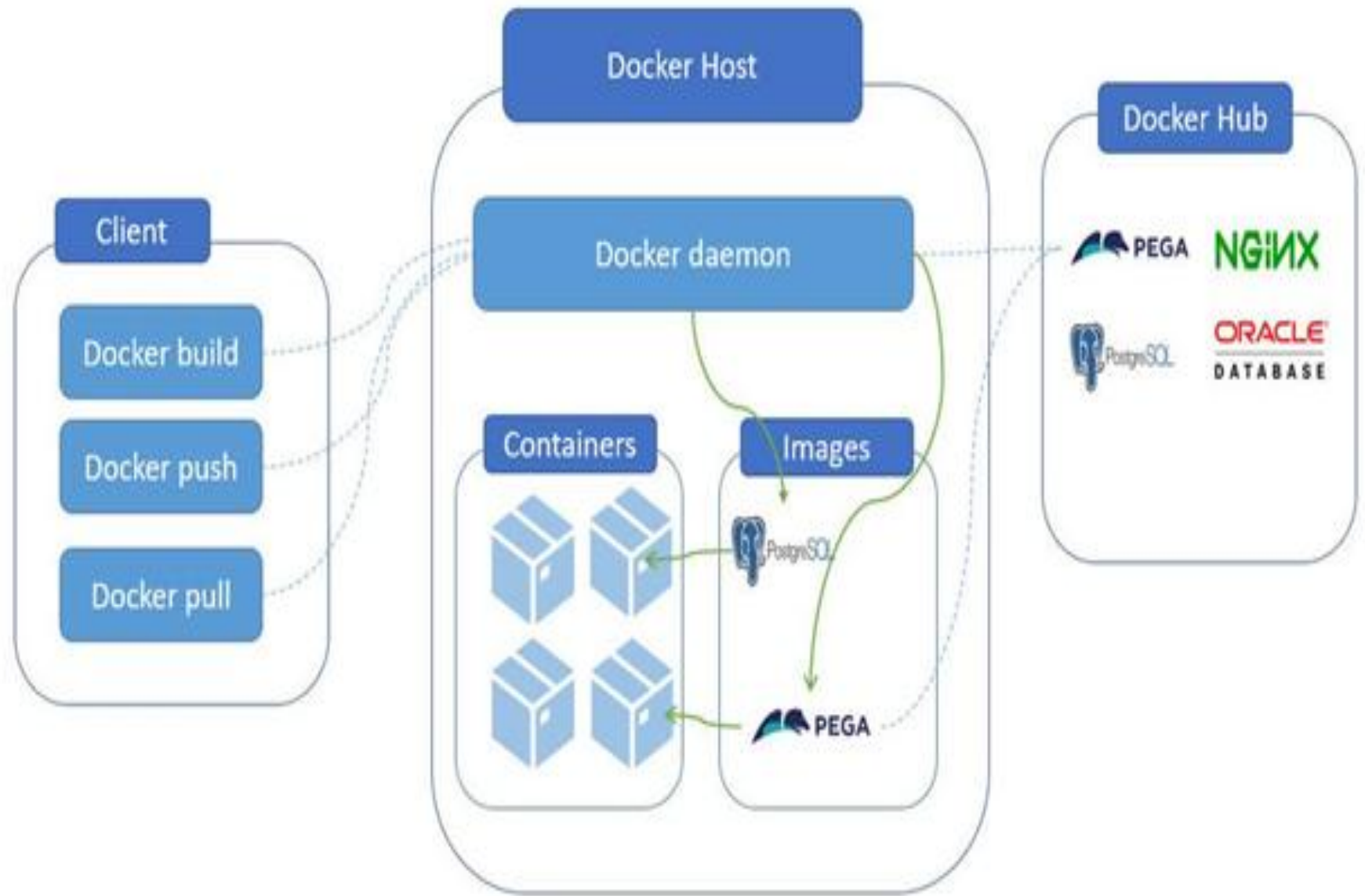
Without Docker



VS

With Docker

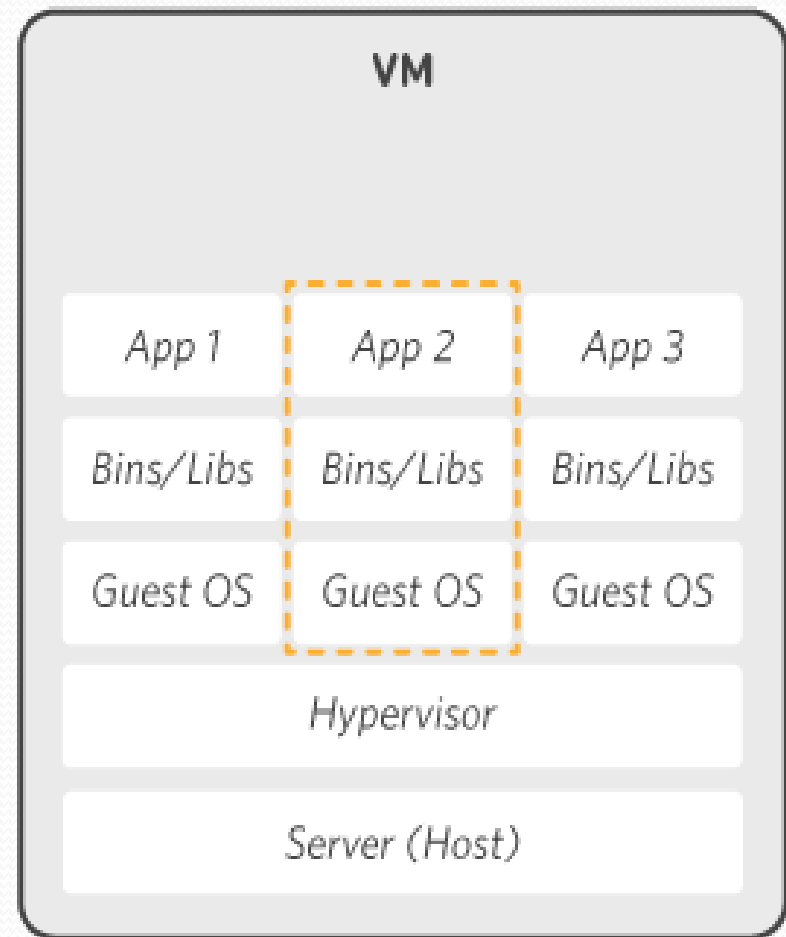
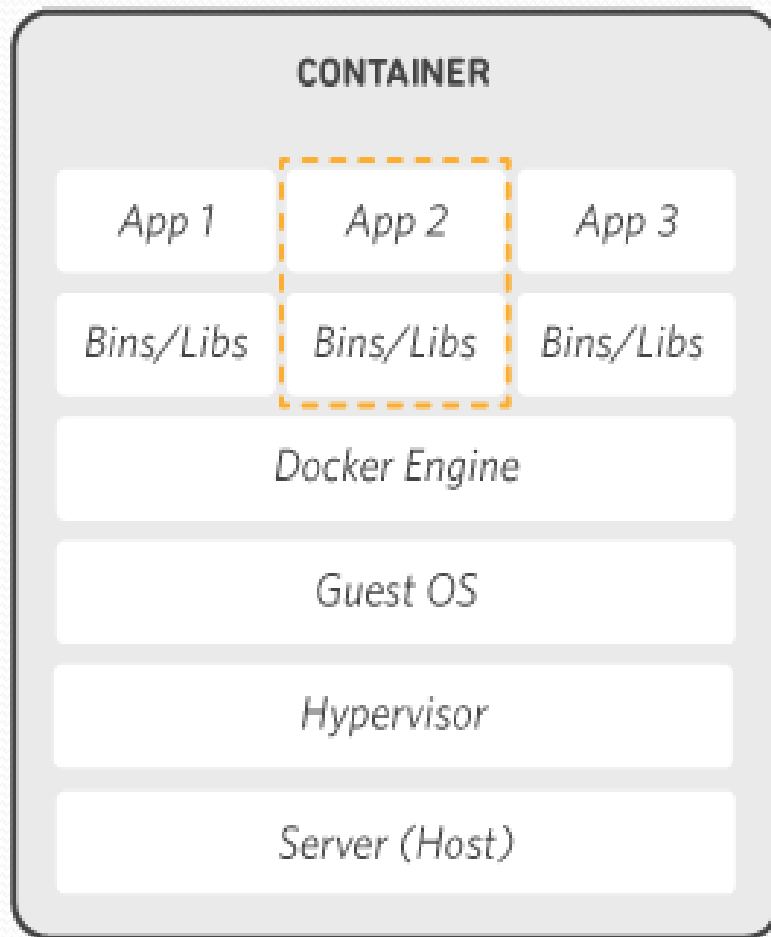




How Docker Works

- Docker works by providing a standard way to run your code. Docker is an operating system for containers.
- Similar to how a virtual machine virtualizes (removes the need to directly manage) server hardware, containers virtualize the operating system of a server.
- Docker is installed on each server and provides simple commands you can use to build, start, or stop containers.
- AWS services such as AWS Fargate, Amazon ECS, Amazon EKS, and AWS Batch make it easy to run and manage Docker containers at scale.

How Docker Works



Why Use Docker

- Using Docker lets you ship code faster, standardize application operations, seamlessly move code, and save money by improving resource utilization.
- With Docker, you get a single object that can reliably run anywhere. Docker's simple and straightforward syntax gives you full control.
- Wide adoption means there's a robust ecosystem of tools and off-the-shelf applications that are ready to use with Docker.